OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science (Code 308)

Data Visualization using Pyplot

Line Charts, Bar Charts, Histograms, Scatter Plots, Customization & Anatomy of a Figure

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Syllabus Section	Core Theoretical & Practical Concepts	CUET Weightage & Exam Focus	Question Typologies
Purpose of Plotting	Role of data visualization; visual perception principles; exploratory data analysis (EDA); Matplotlib architecture; state-based Pyplot interface (`plt`) vs Object-Oriented (`fig, ax = plt.subplots()`).	Moderate (1-2 Questions); foundational conceptual questions on graphical rendering and interface models.	Interface distinction questions, figure anatomy component identification.
Line & Scatter Plots	Drawing line plots via `plt.plot()`; single list input (Y-values with default X RangeIndex); line styles, line widths, and color codes; marker styles and sizing; scatter plots via `plt.scatter()`; coordinate mapping; point sizing `s` and color mapping `c`.	High (2-3 Questions); testing default coordinate generation and marker shorthand syntax.	Output coordinate derivation, shorthand string decomposition (`'r--o'`), parameter validation.
Bar Charts & Histograms	Vertical bar charts (`plt.bar`) and horizontal bar charts (`plt.barh`); multiple bar charts with manual X-coordinate offsets; stacked bar charts using `bottom` and `left`; histograms via `plt.hist()`; bin specifications, range, cumulative, and density parameters.	Very High (3-4 Questions); heavy emphasis on difference between bar chart and histogram, and stacked bar syntax.	Code completion for multiple/stacked bars, bin count calculation, frequency distribution tracing.
Plot Customization & I/O	Adding chart title (`plt.title`), axis labels (`plt.xlabel`, `plt.ylabel`), and legend (`plt.legend`); setting axis limits (`plt.xlim`, `plt.ylim`); tick marks and label rotation (`plt.xticks`, `plt.yticks`); grid activation (`plt.grid`); saving plots via `plt.savefig()` vs displaying via `plt.show()`.	Extremely High (3-4 Questions); tested across all CUET sittings; focus on the savefig-before-show ordering trap.	Order of execution bugs, legend label omission diagnosis, parameter matching.

CUET HIGH-YIELD EXAMINATION INVARIANTS & FATAL TRAPS

1. **The `savefig()` vs `show()` Fatal Ordering Trap:** In Matplotlib's Pyplot state machine, calling `plt.show()` displays the plot in a window and **immediately flushes and clears the current Figure canvas**! If you call `plt.savefig('plot.png')` after `plt.show()`, Matplotlib saves a completely blank white image! You must **always** call `savefig()` before `show()`.
2. **Single List in `plt.plot()`:** If only one sequence is supplied (e.g., `plt.plot([10, 20, 30])`), Matplotlib treats these values strictly as **Y-coordinates**. The X-coordinates are automatically generated as the zero-based integer index `0, 1, 2`.
3. **Stacked Bar Parameter Mismatch:** To create a vertically stacked bar chart, you must pass the `bottom` parameter (e.g., `plt.bar(x, y2, bottom=y1)`). Passing `top` or `stacked=True` raises an immediate `TypeError`. For horizontal stacked bar charts, the parameter is `left`.
4. **Legend Activation Invariant:** Executing `plt.legend()` without having specified `label='...'` inside the plotting functions (`plt.plot`, `plt.bar`) displays an empty legend box or raises a `UserWarning: No artists with labels found to put in legend`.
5. **Bar Chart vs Histogram Distinction:** A Bar Chart compares categorical data with discrete, separated bars. A Histogram displays continuous numerical distributions grouped into contiguous bins with no gaps between bars (unless data is absent in that bin).

Foundations of Data Visualization & Matplotlib Architecture

Data visualization is the graphical representation of information and quantitative data. By translating complex numerical matrices and multi-dimensional tabular datasets into visual primitives—such as lines, bars, geometric markers, and heatmaps—visualization enables data analysts to rapidly identify underlying patterns, non-linear trends, cluster groupings, and statistical outliers that remain obscured in raw tabular spreadsheets.

1. The Matplotlib Ecosystem & Pyplot Interface

Created by John D. Hunter in 2003, **Matplotlib** is the bedrock visualization library in the Python scientific stack. It is structured around an internal hierarchy that mimics the high-level plotting syntax of MATLAB while retaining the full object-oriented power of Python.
Within Matplotlib, the **`pyplot`** module (conventionally imported as `import matplotlib.pyplot as plt`) provides a MATLAB-like, state-based procedural interface. In the state-based paradigm:

- A global state engine implicitly tracks the "current active figure" and "current active axes".
- Any plotting command (such as `plt.plot()` or `plt.title()`) automatically applies its graphical operations to the currently active axes without requiring explicit object referencing.

2. Pyplot Interface vs. Object-Oriented (OO) Interface

While beginner scripts frequently rely on the procedural `plt` interface, production software and complex multi-plot figures use the Object-Oriented interface:

- **Pyplot State-Based Interface:** Simple, procedural, rapid prototyping. Commands act on whichever plot is active:
```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [4, 5, 6])
plt.title("Procedural Plot")
```
- **Object-Oriented (OO) Interface:** Explicit, modular, scalable. Instantiates explicit `Figure` and `Axes` objects:
```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot([1, 2, 3], [4, 5, 6])
ax.set_title("Object-Oriented Plot")
```

Anatomy of a Matplotlib Canvas: Figure vs. Axes

- **Figure (`fig`)**: The top-level bounding canvas or page upon which everything is drawn. It can contain one or multiple subplots (Axes), overall suptitles, and legends.
- **Axes (`ax`)**: The actual plotting area or coordinate system where data is rendered. An Axes contains coordinate spines, X and Y Axis objects, tick marks, labels, and the rendered visual geometry (lines, bars, points).

Line Plots: Construction, Shorthand Notation & Styling

A **Line Plot** connects discrete sequential data points using continuous line segments. It is the gold standard for visualizing time-series trajectories, continuous mathematical functions, and progressive experimental observations.

1. Basic Line Plot Syntax: `plt.plot()`

```
plt.plot(*args, scalex=True, scaley=True, data=None, **kwargs)
```

- **Two-Argument Call:** `plt.plot(x, y)` explicitly maps sequence `x` onto the horizontal axis and sequence `y` onto the vertical axis. Both sequences must possess identical lengths.
- **Single-Argument Call:** `plt.plot(y)` supplies only the dependent variable. Matplotlib automatically generates the independent variable x as an integer sequence: `range(len(y))` $(0, 1, 2, ..., n-1)$.

```
import matplotlib.pyplot as plt

# Single list input: X-coordinates default strictly to 0, 1, 2, 3
temps = [24.5, 26.0, 25.2, 28.1]
plt.plot(temps)
# Plots coordinates: (0, 24.5), (1, 26.0), (2, 25.2), (3, 28.1)
```

2. Formatting Line Color, Style & Thickness

Customizing line aesthetics is achieved via explicit keyword arguments:

- `color` (or `c`): Accepts color names (`'red'`, `'blue'`, `'green'`), hex strings (`'#1E3A8A'`), or standard 1-character abbreviations:
  - `'b'` = blue, `'g'` = green, `'r'` = red, `'c'` = cyan, `'m'` = magenta, `'y'` = yellow, `'k'` = black, `'w'` = white.
- `linestyle` (or `ls`): Defines the stroke pattern of the connecting line:
  - `'-'` or `'solid'` (default solid line)
  - `'--'` or `'dashed'` (dashed stroke)
  - `'-.'` or `'dashdot'` (alternating dash and dot)
  - `':'` or `'dotted'` (series of dots)
  - `'None'` or `' '` (draws no connecting line; shows only markers)
- `linewidth` (or `lw`): Float value specifying stroke thickness in points (default: `1.5`).

3. Marker Customization Protocols

To emphasize the exact coordinate samples along a continuous curve, geometric markers can be overlaid:

- `marker`: Character symbol defining marker geometry:
  - `'o'` = circle, `'s'` = square, `'^'` = triangle up, `'v'` = triangle down, `'D'` = diamond, `'*'` = star, `'+'` = plus, `'x'` = cross, `'.' = point`.
- `markersize` (or `ms`): Float value defining marker diameter in points.
- `markerfacecolor` (or `mfc`): Fill color of the marker interior.
- `markeredgecolor` (or `mec`): Stroke color of the marker boundary.
- `markeredgewidth` (or `mew`): Stroke width of the marker boundary.

4. Compact Shorthand Format String (`[color][marker][line]`)

Matplotlib supports a concise shorthand format string parameter combining color, marker, and linestyle into a single compact token passed as the third positional argument:

```
plt.plot(x, y, 'r--o') # Red color, dashed line, circle markers
plt.plot(x, y, 'b:*')  # Blue color, dotted line, star markers
plt.plot(x, y, 'gs')   # Green color, square markers, NO connecting line
```

Comparative Matrix: Matplotlib Chart Typologies & Calling Signatures

Systematic parameter matrix across Line, Bar, Histogram, and Scatter plots in Pyplot

CUET CODE 308 · HIGH-YIELD MATRIX

Chart Type	Pyplot Function	Primary Analytical Purpose	Key Parameters & Flags	CUET High-Yield Trap
Line Plot	`plt.plot(x, y)`	Display trends, continuous temporal changes, and mathematical curves.	`color`, `linestyle` (`'-'`, `'--'`, `':'`), `linewidth`, `marker`, `markersize`, `label`.	Passing single list `plt.plot([1, 4, 9])` uses it as **Y-values**; X defaults to `0, 1, 2`!
Vertical Bar Chart	`plt.bar(x, height)`	Compare discrete discrete categorical metrics side-by-side.	`width` (default 0.8), `color`, `edgecolor`, `bottom` (for stacking), `align`.	For multiple bar charts, X-coordinates must be manually shifted by bar `width`!
Horizontal Bar Chart	`plt.barh(y, width)`	Compare categories with long text labels that crowd vertical axes.	`height` (default 0.8), `left` (for horizontal stacking), `color`, `edgecolor`.	Arguments are flipped: 1st argument is vertical Y position; 2nd is horizontal `width`.
Histogram	`plt.hist(x)`	Visualize frequency distributions of continuous numerical datasets.	`bins` (default 10 or sequence), `range`, `density=True`, `cumulative=True`, `edgecolor`.	A histogram plots frequency over **binned intervals**, NOT discrete categories like a bar chart!
Scatter Plot	`plt.scatter(x, y)`	Detect correlation, clustering, and relationships between two continuous variables.	`s` (marker size in points^2), `c` (color sequence), `marker`, `alpha` (transparency), `cmap`.	Parameter for size is strictly `s` (not `size`); parameter for color is `c`.

OurMocks CUET UG Series · Computer Science (Code 308) Fig 1: Pyplot Chart Specification Matrix

Figure 1: Fig 1: Comparative Matrix — Pyplot Chart Typologies, Calling Signatures, Parameter Semantics, and CUET Traps.

Categorical Comparisons: Vertical & Horizontal Bar Charts

A **Bar Chart** represents categorical variables using rectangular bars where the length or height of each bar is proportional to the numerical magnitude it represents.

1. Vertical Bar Charts: `plt.bar()`

```
plt.bar(x, height, width=0.8, bottom=None, align='center', data=None, **kwargs)
```

- `x`: Coordinates of the bar positions along the horizontal axis (categories or numeric offsets).
- `height`: The scalar magnitudes determining the vertical heights of the bars.
- `width`: Float value defining bar thickness (default: `0.8`).
- `color`: Bar fill color.
- `edgecolor`: Border stroke color around bars.
- `bottom`: The vertical starting coordinate for the base of the bars (crucial for stacked bar charts).

2. Horizontal Bar Charts: `plt.barh()`

When categorical labels are lengthy strings (such as company names or book titles), vertical bar charts cause severe text crowding along the horizontal axis. A horizontal bar chart flips the coordinate orientation:

```
plt.barh(y, width, height=0.8, left=None, align='center', **kwargs)
```

- `y`: Vertical positions of the bars.
- `width`: The horizontal lengths of the bars.
- `height`: The thickness of each horizontal bar (default: `0.8`).

- `left`: The starting horizontal offset for the base of the bars (used for horizontal stacked bars).

3. Multiple Bar Charts (Grouped Side-by-Side)

To display multiple categorical metrics side-by-side (e.g., comparing Girls vs. Boys test scores across departments), the X-coordinates must be explicitly shifted:

```
import numpy as np
import matplotlib.pyplot as plt

categories = ['CS', 'IT', 'EC', 'ME']
x_indices = np.arange(len(categories))  # array([0, 1, 2, 3])
bar_width = 0.35

boys_scores = [85, 78, 92, 88]
girls_scores = [90, 84, 95, 86]

# Plotting Boys shifted left, Girls shifted right
plt.bar(x_indices - bar_width/2, boys_scores, width=bar_width, label='Boys', color='navy')
plt.bar(x_indices + bar_width/2, girls_scores, width=bar_width, label='Girls', color='teal')

# Replacing numeric offsets with category labels
plt.xticks(ticks=x_indices, labels=categories)
plt.legend()
```

4. Stacked Bar Charts

In a stacked bar chart, multiple segments are placed directly on top of each other to show both individual contributions and the combined aggregate total.

- To stack Series 2 on top of Series 1, set `bottom=Series1_values` in the second `plt.bar()` call!

```
sem1_pass = [45, 52, 48]
sem2_pass = [38, 44, 42]
colleges = ['ColA', 'ColB', 'ColC']

plt.bar(colleges, sem1_pass, label='Sem 1', color='royalblue')
# bottom parameter elevates Sem 2 above Sem 1
plt.bar(colleges, sem2_pass, bottom=sem1_pass, label='Sem 2', color='orange')
plt.legend()
```

Continuous Frequency Distributions: Histograms

A **Histogram** is an accurate graphical representation of the distribution of numerical continuous data. Unlike a bar chart (which plots discrete categories against a summary metric), a histogram groups continuous observations into contiguous intervals termed **bins** and plots the count (frequency) of values falling into each bin.

1. Syntax of `plt.hist()`

```
plt.hist(
    x,
    bins=None,
    range=None,
    density=False,
    cumulative=False,
    bottom=None,
    histtype='bar',
    align='mid',
    orientation='vertical',
    rwidth=None,
    color=None,
    edgecolor=None
)
```

- `x`: The 1D array or sequence of continuous numerical values to be analyzed.
- `bins`: Specifies how the data range is partitioned:
  - An integer k: Divides the span into k equal-width bins (default: `bins=10`).
  - A sequence of bin edges: Explicitly defines custom interval boundaries (e.g., `bins=[0, 20, 50, 100]`).
- `cumulative`: If `True`, computes a cumulative frequency histogram where each bin accumulates the counts of all preceding bins.
- `density`: If `True`, normalizes the bin counts so that the total area under the histogram integrates to 1 (probability density).
- `edgecolor`: Distinct stroke color separating adjacent bins (e.g., `edgecolor='black'`).

```
marks = [55, 62, 75, 78, 82, 85, 88, 91, 92, 95, 98, 42, 67, 73]
# Explicit 4 bins: [40, 60), [60, 80), [80, 90), [90, 100]
plt.hist(marks, bins=[40, 60, 80, 90, 100], color='skyblue', edgecolor='black')
```

CUET FUNDAMENTAL DISTINCTION: BAR CHART VS. HISTOGRAM

- **Bar Chart:** X-axis represents **discrete categories** (e.g., 'Delhi', 'Mumbai'). Gaps exist between bars to convey independence. Bar heights represent a metric value (e.g., Revenue).
- **Histogram:** X-axis represents a **continuous numeric scale** divided into adjacent bins (e.g., ages 10-20, 20-30). Bars touch each other without gaps. Bar area represents frequency of observations.

Bivariate Correlation: Scatter Plots

A **Scatter Plot** plots individual data points as unjoined coordinates on a Cartesian plane, where the horizontal position is governed by one variable (x) and the vertical position by another (y). It is the primary tool for analyzing correlation, cluster separation, and dispersion.

Syntax of `plt.scatter()`

```
plt.scatter(
    x, y,
    s=None,
    c=None,
    marker=None,
    cmap=None,
    norm=None,
    vmin=None,
    vmax=None,
    alpha=None,
    linewidths=None,
    edgecolors=None
)
```

- `x`, `y`: Data coordinate arrays of identical length.
- `s`: Marker size in points squared (points^2). Can be a single scalar or an array of sizes matching `len(x)`.
- `c`: Marker color. Can be a single color string or a sequence of continuous values mapped via `cmap`.
- `alpha`: Float ranging from 0.0 (completely transparent) to 1.0 (completely opaque). Crucial for mitigating visual occlusion in overlapping data clusters.

```
study_hours = [2, 3, 5, 7, 8, 10]
exam_marks = [50, 58, 72, 85, 90, 98]

# Visualizing positive correlation
plt.scatter(study_hours, exam_marks, color='crimson', marker='^', s=80, alpha=0.85)
```

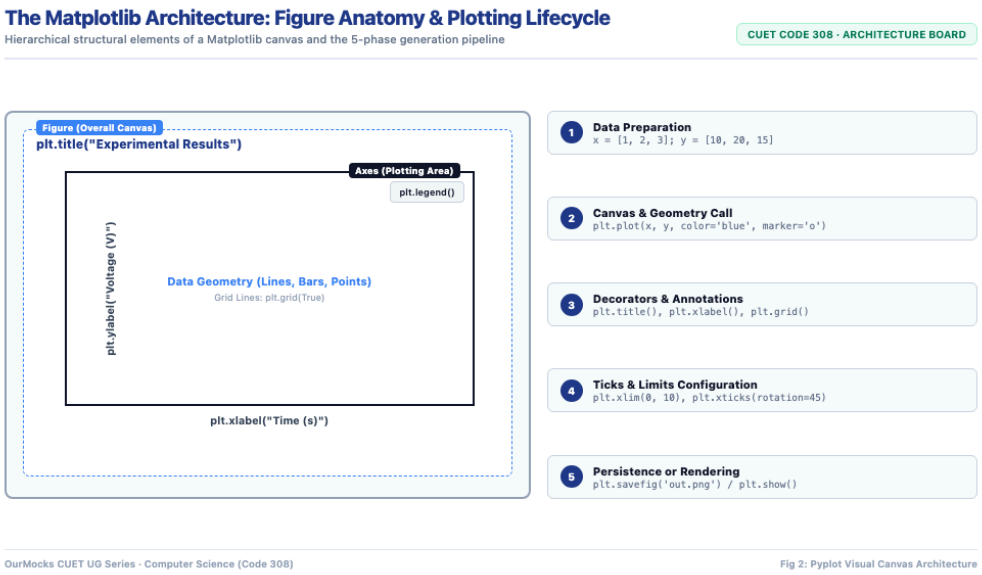


Figure 2: Fig 2: Pyplot Visual Canvas Architecture — Anatomy of a Figure, Component Hierarchy, and Generation Lifecycle.

Plot Decorators, Canvas Customization & Axis Controls

To transform a raw geometric shape into a publication-grade, self-describing academic visualization, Pyplot provides comprehensive annotation primitives.

1. Chart Titles, Axis Labels & Text Formatting

- `plt.title(label, fontdict=None, loc='center', pad=None)`: Sets the primary canvas title. `loc` can be `'center'`, `'left'`, or `'right'`.
- `plt.xlabel(xlabel, fontdict=None, labelpad=None)`: Labels the horizontal axis.
- `plt.ylabel(ylabel, fontdict=None, labelpad=None)`: Labels the vertical axis.

2. Legend Activation & Positioning: `plt.legend()`

A legend identifies distinct data series rendered on the same canvas:

- **Prerequisite:** Each plotting call must supply an explicit `label` parameter (e.g., `plt.plot(x, y, label='2024')`).
- `plt.legend(loc='best', frameon=True, shadow=True, fontsize=10)`
- `loc` placement options: `'best'` (default, automatically detects least crowded area), `'upper right'`, `'upper left'`, `'lower left'`, `'lower right'`, `'center'`.

3. Ticks, Tick Labels & Axis Limits

- `plt.xlim(xmin, xmax)` and `plt.ylim(ymin, ymax)`: Constrain or expand the visible coordinate boundaries of the axes.
- `plt.xticks(ticks=None, labels=None, rotation=None)`: Manages horizontal tick positions and text. Setting `rotation=45` or `rotation=90` tilts tick text to prevent label overlap.
- `plt.yticks(ticks=None, labels=None)`: Configures vertical tick intervals.

4. Background Grid: `plt.grid()`

- `plt.grid(visible=True, which='major', axis='both', color='gray', linestyle='--', linewidth=0.5)`
- `axis`: `'both'` (default), `'x'`, or `'y'` to display vertical or horizontal gridlines selectively.

File Output Persistence & The Pyplot State Machine

Exporting visualizations to disk requires understanding the interaction between `plt.show()` and `plt.savefig()`.

1. Saving Figures to File: `plt.savefig()`

```
plt.savefig(fname, dpi=None, format=None, bbox_inches=None, pad_inches=0.1, transparent=False)
```

- `fname`: File path string (e.g., `'output_chart.png'`).
- `dpi`: Dots per inch (resolution). Standard web resolution is 100-150 dpi; publication print resolution is 300 dpi.
- `format`: Target file extension: `'png'`, `'pdf'`, `'svg'`, `'jpeg'`, `'eps'`. If omitted, inferred from `fname`.
- `bbox_inches='tight'`: Automatically crops excess whitespace padding around the canvas edges.

THE GOLDEN PYPLOT INVARIANT: SAVEFIG() BEFORE SHOW()

In Matplotlib's procedural state machine, `plt.show()` opens the GUI event loop, renders the figure window, and **immediately flushes and clears the Figure canvas buffer** upon completion!

```
plt.plot(x, y)
plt.show()              # Canvas is flushed and reset!
plt.savefig('test.png')  # BUG: Saves an entirely BLANK white canvas!
```

Correct Protocol: Always invoke `plt.savefig()` **prior** to `plt.show()`!

Comprehensive CUET Code Traces & Analytical Walkthroughs

MULTIPLE BAR CHART OFFSET & COORDINATE DERIVATION

Problem: Consider the following script intended to generate side-by-side comparison bars:

```
import numpy as np
import matplotlib.pyplot as plt

sections = ['Sec A', 'Sec B', 'Sec C']
pos = np.arange(len(sections))
w = 0.4

math_avg = [75, 82, 90]
sci_avg = [80, 78, 85]

plt.bar(pos - w/2, math_avg, width=w, label='Math')
plt.bar(pos + w/2, sci_avg, width=w, label='Science')
plt.xticks(pos, sections)
```

1. What are the numerical center coordinates of the two bars for `'Sec B'`?
2. What is the total combined width occupied by both bars for any given section?
3. What is the output of `len(plt.xticks()[0])`?

Solution:

- **Part 1:** For `'Sec B'`, index `pos[1] == 1`.
  - Math bar center: $\text{pos}[1] - \frac{w}{2} = 1 - \frac{0.4}{2} = 1 - 0.2 = 0.8$.
  - Science bar center: $\text{pos}[1] + \frac{w}{2} = 1 + \frac{0.4}{2} = 1 + 0.2 = 1.2$.

$$\text{Math center} = 0.8, \quad \text{Science center} = 1.2$$

- **Part 2:** Each bar has width 0.4. Placed side-by-side:
$$\text{Total Width} = 0.4 + 0.4 = 0.8$$
- **Part 3:** There are 3 tick marks corresponding to `pos` $(0, 1, 2)$.
$$\text{len(plt.xticks()[0])} = 3$$

HISTOGRAM BIN BOUNDARIES & FREQUENCY EXTRACTION TRACE

Problem: An experiment measures signal latency values:

```
import matplotlib.pyplot as plt
latencies = [12, 18, 25, 29, 32, 38, 45, 48, 52, 59]
n, bins, patches = plt.hist(latencies, bins=[10, 30, 50, 70], edgecolor='k')
```

1. How many bins are generated, and what are their exact intervals?
2. What is the array of frequencies returned in variable `n`?
3. If `cumulative=True` was set, what would be the value of the final entry in `n`?

Solution:

- **Part 1:** A bin boundary sequence of length k defines $k - 1$ bins. The intervals are: $[10, 30)$, $[30, 50)$, $[50, 70]$ (3 bins total).
- **Part 2:** Counting observations in each interval:
  - Bin 1 $[10, 30)$: Values are $12, 18, 25, 29$ (4 values).
  - Bin 2 $[30, 50)$: Values are $32, 38, 45, 48$ (4 values).
  - Bin 3 $[50, 70]$: Values are $52, 59$ (2 values).
$$n = \text{array}([4., 4., 2.])$$
- **Part 3:** In a cumulative histogram, the final bin contains the total count of all samples $(4 + 4 + 2 = 10)$.
$$\text{Final entry} = 10.0$$

OurMocks

Multi-Plot Canvas Architectures: Subplots & Grid Layouts

Real-world analytical dashboards and exploratory research workflows frequently require comparing multiple disparate charts side-by-side within a single overall figure canvas. Matplotlib provides two complementary mechanisms for multi-plot layout management: procedural subplots (`plt.subplot()`) and object-oriented subplot grids (`plt.subplots()`).

1. Procedural Subplots: `plt.subplot(nrows, ncols, index)`

The `plt.subplot()` function divides the current Figure into an imaginary grid of rows and columns and activates a specific cell for subsequent plotting calls:

```
plt.subplot(nrows, ncols, index)
```

- `nrows`: Total number of rows in the virtual grid.
- `ncols`: Total number of columns in the virtual grid.
- `index`: 1-based integer index specifying which cell is active. Numbering starts at 1 at the top-left cell and increases row-by-row from left to right.
- **Three-Digit Shorthand:** If all three integer arguments are single digits (< 10), commas can be omitted: `plt.subplot(2, 2, 1)` is completely identical to `plt.subplot(221)`.

```
import matplotlib.pyplot as plt

# 2x1 Grid: Two plots stacked vertically
plt.subplot(2, 1, 1)
plt.plot([1, 2, 3], [10, 20, 15], 'r-o')
plt.title("Voltage Trajectory")

plt.subplot(2, 1, 2)
plt.plot([1, 2, 3], [0.5, 0.8, 0.6], 'b--s')
plt.title("Current Trajectory")

plt.tight_layout()  # Automatically adjusts subplot spacing to prevent overlapping titles
```

2. Object-Oriented Subplot Grids: `plt.subplots()`

In the Object-Oriented paradigm, `plt.subplots()` instantiates both the top-level `Figure` container and the full array of `Axes` objects in a single concise call:

```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 8), sharex=True, sharey=True)
```

- `sharex=True` / `sharey=True`: Ties coordinate limits across subplots so that zooming or scaling one axis automatically applies to all adjacent axes.
- **Dimensionality of `axes`:**
  - If nrows $= 1$ and ncols $= 1$: Returns a single scalar `AxesSubplot` object.
  - If nrows $= 1$ or ncols $= 1$: Returns a 1D NumPy array of `Axes` objects (`axes[0]`, `axes[1]`).
  - If nrows > 1 and ncols > 1: Returns a 2D NumPy array indexed as `axes[row_index, col_index]`.
- **Global Figure Title:** In multi-plot figures, individual subplot titles are assigned via `ax.set_title()`, while the overall banner title across the entire Figure is created using `fig.suptitle('Overall Experiment Title')`.

Proportional Composition: Pie Charts in Pyplot

A **Pie Chart** represents numerical proportions as angular slices of a circular statistical graphic. The arc length, central angle, and area of each slice are strictly proportional to the quantity it represents relative to the whole (360°).

1. Syntax of `plt.pie()`

```
plt.pie(
    x,
    explode=None,
    labels=None,
    colors=None,
    autopct=None,
    pctdistance=0.6,
    shadow=False,
    labeldistance=1.1,
    startangle=0,
    radius=1,
    counterclock=True,
    wedgeprops=None,
    textprops=None,
    center=(0, 0),
    frame=False
)
```

- `x`: 1D array of non-negative numerical quantities. Matplotlib normalizes the values: if sum(x) $\neq 1$, each slice's proportion is calculated as $\frac{x_i}{\sum x}$.
- `labels`: Sequence of strings providing category names positioned outside each slice.
- `autopct`: String formatting template or callable function used to label the numeric percentage inside each wedge:
  - `'%1.1f%%'`: Displays percentage with 1 decimal place (e.g., `28.5%`).

- `'%1.0f%%'`: Displays integer rounded percentage (e.g., `29%`).
- `'%1.2f%%'`: Displays percentage with 2 decimal places (e.g., `28.52%`).

- `explode`: Sequence of numerical offsets specifying how far each wedge is shifted radially outward from the circle center (useful for emphasizing a focal category):
  - `explode=[0, 0.1, 0, 0]` extracts the second slice outward by 10% of the radius.
- `startangle`: Float angle in degrees from which the first slice begins (measured counter-clockwise from the positive horizontal X-axis). Common setting is `startangle=90` (starts pointing vertically upwards).
- `shadow`: Boolean (`True` or `False`). If `True`, draws a drop shadow underneath the pie wedge geometry.

```
import matplotlib.pyplot as plt

expenses = [12000, 5000, 8000, 3000, 2000]
categories = ['Rent', 'Groceries', 'Utilities', 'Entertainment', 'Misc']
slice_offsets = [0.1, 0, 0, 0, 0]  # Explode 'Rent'

plt.pie(
    expenses,
    labels=categories,
    autopct='%1.1f%%',
    explode=slice_offsets,
    shadow=True,
    startangle=140,
    colors=['gold', 'yellowgreen', 'lightcoral', 'lightskyblue', 'violet']
)
plt.title("Monthly Expenditure Breakdown")
```

CUET HIGH-YIELD DETAIL: ASPECT RATIO IN PIE CHARTS

To guarantee that a pie chart renders as a true circle rather than an elliptical egg shape on screens with non-square pixel dimensions, invoke `plt.axis('equal')` immediately before displaying or saving the plot!

Canvas Annotations: Point Annotations & Reference Lines

Transforming exploratory charts into explanatory figures requires drawing contextual indicators, threshold spans, and data callouts.

1. Callout Annotations: `plt.annotate()`

The `plt.annotate()` function renders text paired with an optional pointer arrow directed at an exact coordinate feature:

```
plt.annotate(
    text='Peak Outlier',
    xy=(4, 98),              # Coordinate point being pointed to (arrow tip)
    xytext=(5, 80),          # Coordinate position of the text label
    arrowprops=dict(facecolor='black', shrink=0.05, width=1, headwidth=6)
)
```

2. Horizontal & Vertical Reference Spans: `axhline()` & `axvline()`

- `plt.axhline(y=0, color='r', linestyle='--', linewidth=1)`: Draws an infinite horizontal benchmark line across the entire plot at height y.
- `plt.axvline(x=5, color='g', linestyle=':', linewidth=1)`: Draws an infinite vertical threshold line at coordinate x.
- `plt.axhspan(ymin=40, ymax=60, color='yellow', alpha=0.2)`: Draws a shaded horizontal background region spanning from `ymin` to `ymax`.

Direct Visualization from Pandas DataFrames

Modern data science workflows rarely manually extract lists from DataFrames before plotting. Pandas provides built-in `.plot()` wrapper methods directly on Series and DataFrame objects that delegate rendering directly to Matplotlib.

```
# Direct Pandas plotting interface
df.plot(
    x='Quarter',
    y=['Revenue', 'Expenses'],
    kind='bar',          # 'line', 'bar', 'barh', 'hist', 'scatter', 'pie'
    title='Quarterly Performance',
    figsize=(8, 5),
    grid=True
)
```

- Calling `df.plot(kind='line')` automatically maps the DataFrame row index onto the X-axis and creates a separate colored line for every numeric column in the table, complete with an automatic legend!

SUBPLOT GRID INDEXING & CANVAS CELL NAVIGATION

Problem: Consider the following procedural plotting commands:

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))
```

```
plt.subplot(3, 2, 4)
plt.plot([1, 2], [3, 4])
plt.subplot(3, 2, 5)
plt.plot([5, 6], [7, 8])
```

1. What is the total number of subplots that can be accommodated in this grid?
2. In which row and column is the subplot created by `plt.subplot(3, 2, 4)` located?
3. In which row and column is `plt.subplot(3, 2, 5)` located?

Solution:

- **Part 1:** The grid is defined with nrows = 3 and ncols = 2.
$$\text{Total subplots} = 3 \times 2 = 6$$
- **Part 2:** Cells are numbered row-by-row:
  - Row 1 contains cells 1 and 2.
  - Row 2 contains cells 3 and 4.
  - Row 3 contains cells 5 and 6.

  Therefore, cell 4 is located at:
$$\text{Row 2, Column 2}$$
- **Part 3:** Cell 5 is the first cell in the third row:
$$\text{Row 3, Column 1}$$

PIE CHART PERCENTAGE & CENTRAL ANGLE DERIVATION

Problem: A market survey records consumer smartphone operating system preferences across 400 respondents:

```
import matplotlib.pyplot as plt
counts = [240, 120, 32, 8]
labels = ['Android', 'iOS', 'Windows', 'Other']
plt.pie(counts, labels=labels, autopct='%1.1f%%')
```

1. What percentage is displayed for `'Android'` and `'iOS'`?
2. What is the central angle (in degrees) occupied by the `'Other'` slice?
3. If `explode=[0.1, 0, 0, 0]` is applied, which slice is exploded outward?

Solution:

- **Part 1:** Total respondents $= 240 + 120 + 32 + 8 = 400$.
$$\text{Android } \% = \frac{240}{400} \times 100 = 60.0\%$$
$$\text{iOS } \% = \frac{120}{400} \times 100 = 30.0\%$$
- **Part 2:** The central angle is proportional to the fraction of 360 degrees:
$$\text{Other Angle} = \frac{8}{400} \times 360^\circ = 0.02 \times 360^\circ = 7.2^\circ$$
- **Part 3:** The first offset in `explode` (0.1) corresponds to the first element in `counts`, which is `'Android'`.

Master Synthesis Mindmap: Data Visualization with Pyplot

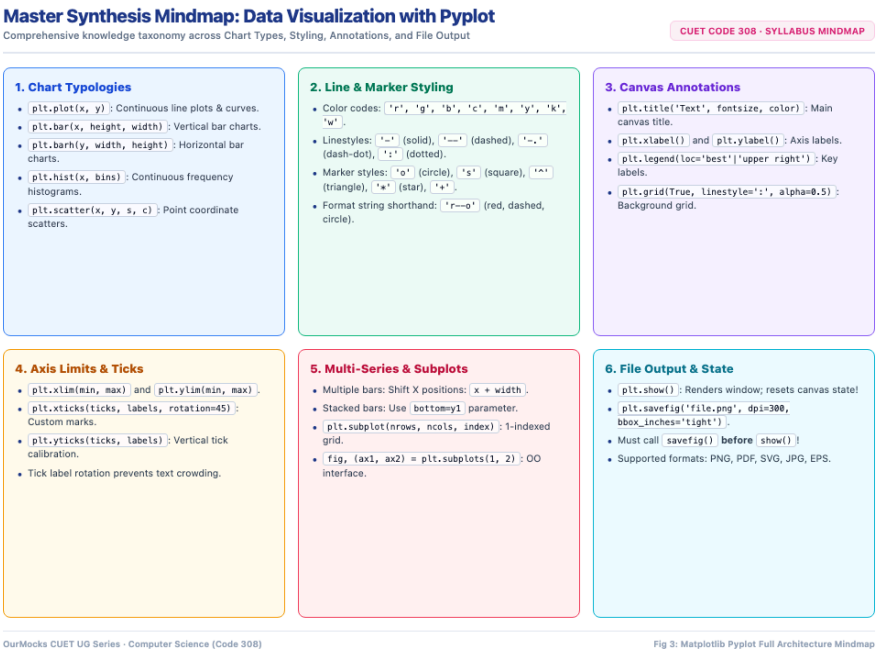


Figure 3: Fig 3: Synthesizer Mindmap — Matplotlib Architecture, Chart Typologies, Styling Properties, and Canvas Annotations.

OurMocks

Cheatsheet & The Seven Fatal CUET Traps

MATPLOTLIB PYPLOT CORE FUNCTIONS & PARAMETERS CHEATSHEET

- **Chart Plotting Primitives:**
  - `plt.plot(x, y, 'fmt', color='r', linestyle='--', linewidth=2, marker='o', markersize=6, label='str')`
  - `plt.bar(x, height, width=0.8, bottom=None, color='b', edgecolor='k', label='str')`
  - `plt.barh(y, width, height=0.8, left=None, color='g', edgecolor='k')`
  - `plt.hist(x, bins=10|[edges], range=(min, max), density=False, cumulative=False, edgecolor='k')`
  - `plt.scatter(x, y, s=50, c='color'|[vals], marker='^', alpha=0.7, cmap='viridis')`
- **Canvas Decorators & Annotations:**
  - `plt.title('Title', fontsize=14, loc='center'|'left'|'right')`
  - `plt.xlabel('Label', fontsize=12)` / `plt.ylabel('Label', fontsize=12)`
  - `plt.legend(loc='best'|'upper right'|'lower left', frameon=True)`
  - `plt.grid(visible=True, axis='both'|'x'|'y', linestyle=':', linewidth=0.5)`
- **Axis Scales, Ticks & Boundaries:**
  - `plt.xlim(min, max)` / `plt.ylim(min, max)`
  - `plt.xticks(ticks=[0,1,2], labels=['A','B','C'], rotation=45)`
  - `plt.yticks(ticks=[...], labels=[...])`
- **Figure Output & Management:**
  - `plt.savefig('fname.png', dpi=300, bbox_inches='tight', format='png')`
  - `plt.show()` (Flushes and clears canvas!)
  - `plt.clf()` (Clears current figure) / `plt.close()` (Closes window)

THE SEVEN FATAL CUET TRAPS IN DATA VISUALIZATION WITH PYPLOT

1. **Trap 1: The `savefig()` Post-`show()` Blank File Trap:** Invoking `plt.show()` renders the GUI window and flushes the buffer. Calling `plt.savefig()` afterwards outputs a completely blank image. Always call `savefig()` before `show()`!
2. **Trap 2: Stacked Bar `bottom` Misspelling:** Vertical stacked bar charts require `bottom=series1`. Writing `stacked=True` or `below=...` raises a fatal `TypeError`! For horizontal bars, use `left=...`.
3. **Trap 3: Single List in `plt.plot()` Argument:** `plt.plot([5, 10, 15])` plots the values as Y-coordinates. The X-coordinates are automatically generated as `0, 1, 2` (RangeIndex), NOT `1, 2, 3`!
4. **Trap 4: Missing `label` in `plt.legend()`:** Calling `plt.legend()` without having specified `label='...'` inside plotting commands produces an empty legend or throws a warning.
5. **Trap 5: Multiple Bar Coordinate Overlap:** Forgetting to shift X-coordinates in multiple bar charts causes bars to be rendered directly on top of each other, obscuring earlier bars!
6. **Trap 6: Scatter Plot Marker Size Parameter:** In `plt.scatter()`, marker size is specified by parameter `s` (points squared), NOT `size` or `ms`! Passing `size=50` causes a `TypeError`.
7. **Trap 7: `plt.barh()` Argument Inversion:** In `plt.barh(y, width)`, the first argument is vertical Y position; the second is horizontal width. Misordering them flips the axes!

CUET Focused Multiple Choice Questions (Part 1)

Q1. A student executes the statement `plt.plot([2, 5, 8, 12])` without specifying a second sequence. How does Matplotlib interpret this single argument?

(A) The values are treated as Y-coordinates; X-coordinates are automatically generated as `[0, 1, 2, 3]`
(B) The values are treated as X-coordinates; Y-coordinates are automatically generated as `[0, 1, 2, 3]`
(C) A `TypeError` is raised because `plt.plot()` strictly requires two positional arguments `(x, y)`
(D) The values are treated as coordinate pairs: `(2, 5)` and `(8, 12)`

Sol. Q1. When a single sequence is passed to `plt.plot()`, Matplotlib treats it as Y-values. The corresponding X-values are automatically generated as zero-based integer indices: $0, 1, ..., n-1$. Here, X is `[0, 1, 2, 3]`.
Answer: (A)

Q2. Which Pyplot function is used to create a horizontal bar chart?

(A) `plt.bar(orientation='horizontal')`
(B) `plt.barh()`
(C) `plt.hbar()`
(D) `plt.plot(kind='barh')`

Sol. Q2. In Matplotlib Pyplot, horizontal bar charts are constructed using the dedicated function `plt.barh(y, width)`.
Answer: (B)

Q3. To draw a vertically stacked bar chart displaying sales of Product B placed directly on top of Product A, which parameter must be supplied to the second `plt.bar()` call?

(A) `stacked=True`
(B) `top=sales_A`
(C) `bottom=sales_A`
(D) `align='top'`

Sol. Q3. In `plt.bar()`, the `bottom` parameter specifies the starting vertical base coordinate of each bar. Setting `bottom=sales_A` elevates Product B so that it stacks directly above Product A.
Answer: (C)

Q4. Which of the following correctly describes the fundamental visual difference between a Bar Chart and a Histogram?

(A) Bar charts display continuous data; histograms display categorical data
(B) Bar charts require sorted data; histograms do not require sorted data
(C) Bar charts can only be vertical; histograms can only be horizontal
(D) Bar charts display discrete categorical variables with separated bars; histograms display continuous distributions divided into contiguous bins

Sol. Q4. A Bar Chart compares discrete categorical variables with gaps between bars. A Histogram represents the frequency distribution of a continuous numeric variable grouped into adjacent, contiguous intervals (bins) without gaps.
Answer: (D)

Q5. What visual styling is specified by the format shorthand string `'r--^'` in `plt.plot(x, y, 'r--^')`?

(A) Red color, dashed line style, triangle-up marker
(B) Red color, dotted line style, square marker
(C) Right-aligned line, double width, circle marker
(D) Rainbow gradient, solid line, triangle-up marker

Sol. Q5. In Pyplot format strings: `'r'` denotes the color Red; `'--'` denotes a Dashed line style; `'^'` denotes a Triangle-up marker symbol.
Answer: (A)

CUET Focused Multiple Choice Questions (Part 2)

Q6. Consider the code snippet:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [10, 20, 30])
plt.show()
plt.savefig('my_plot.png')
```

What is the outcome of the saved image file `'my_plot.png'`?

(A) The file contains the line plot with 300 dpi resolution
(B) The file is completely blank because `plt.show()` flushes and clears the current Figure canvas
(C) A `FileNotFoundError` is thrown by the operating system
(D) The plot is saved twice: once as PNG and once as PDF

Sol. Q6. `plt.show()` renders the plot in an interactive window and immediately flushes the active Figure canvas. Calling `plt.savefig()` after `plt.show()` captures the cleared, empty canvas, resulting in a blank white image. `savefig()` must be called before `show()`.

Answer: (B)

Q7. In `plt.scatter(x, y)`, which parameter explicitly specifies the marker size?

(A) `markersize`
(B) `ms`
(C) `s`
(D) `size`

Sol. Q7. In `plt.scatter()`, marker size is controlled strictly by parameter `s` (points squared). In contrast, `plt.plot()` uses `markersize` or `ms`.

Answer: (C)

Q8. What is the prerequisite for `plt.legend()` to successfully display an explanatory key on the plot?

(A) The plot title must be defined using `plt.title()`
(B) The X and Y axes must both have numerical data
(C) The grid must be enabled using `plt.grid(True)`
(D) Each plotted series must have an assigned `label` parameter inside the plotting call

Sol. Q8. `plt.legend()` gathers all plot artists that possess a non-empty `label` attribute. If no `label` was provided during plotting, the legend cannot identify the data series.

Answer: (D)

Q9. To prevent text overlap when category names on the horizontal axis are long strings, which function and argument should be used?

(A) `plt.xticks(rotation=45)`
(B) `plt.xlabel(wrap=True)`
(C) `plt.xlim(orientation='vertical')`
(D) `plt.axis(rotate=True)`

Sol. Q9. `plt.xticks(rotation=45)` tilts the X-axis tick labels at a 45-degree angle, preventing overlapping text when labels are wide or numerous.

Answer: (A)

Q10. A dataset has values ranging from 5 to 95. The histogram is created with `plt.hist(data, bins=[0, 25, 50, 75, 100])`. How many bins are generated?

(A) 5 bins
(B) 4 bins
(C) 10 bins
(D) 100 bins

Sol. Q10. When a list of edge boundaries is passed to `bins`, the number of bins is equal to `len(bins) - 1`. Here, 5 boundary edges define exactly 4 bins: $[0, 25)$, $[25, 50)$, $[50, 75)$, $[75, 100]$.

Answer: (B)

CUET Focused Multiple Choice Questions (Part 3)

Q11. When creating a grouped side-by-side multiple bar chart for two datasets using `plt.bar()`, why is it necessary to shift the X positions of the bars?

(A) Because Matplotlib will crash if two bars share the same X position
(B) To change the color palette of the second dataset automatically
(C) To prevent the second set of bars from overlapping and occluding the first set of bars
(D) To convert the bars into a histogram distribution

Sol. Q11. If both series are plotted at identical X-coordinates, the second bar chart will draw directly on top of the first, completely obscuring it. Shifting X positions (e.g., $x - \frac{w}{2}$ and $x + \frac{w}{2}$) displays them side-by-side.
Answer: (C)

Q12. Which parameter controls the transparency of points in a scatter plot or lines in a line chart?

(A) `transparent`
(B) `opacity`
(C) `density`
(D) `alpha`

Sol. Q12. In Matplotlib, transparency is controlled by the `alpha` parameter, which accepts a float from 0.0 (completely transparent/invisible) to 1.0 (completely opaque).
Answer: (D)

Q13. In the Object-Oriented interface, what does the statement `fig, ax = plt.subplots(2, 2)` return in variable `ax`?

(A) A 2×2 NumPy array of `Axes` objects indexed as `ax[row, col]`
(B) A single `Axes` object shared across four quadrants
(C) A list of two `Figure` objects
(D) A tuple containing four integer axis coordinates

Sol. Q13. `plt.subplots(2, 2)` creates a grid with 2 rows and 2 columns of subplots. The returned `ax` is a 2-dimensional NumPy array of `AxesSubplot` instances accessible via 2D coordinate indexing `ax[0, 0]`, `ax[0, 1]`, `ax[1, 0]`, and `ax[1, 1]`.
Answer: (A)

Q14. What is the purpose of calling `plt.xlim(0, 100)`?

(A) It divides the horizontal axis into 100 equal tick marks
(B) It sets the view limits of the horizontal X-axis from a minimum of 0 to a maximum of 100
(C) It limits the maximum number of data points to 100
(D) It scales the figure width to 100 inches

Sol. Q14. `plt.xlim(xmin, xmax)` sets the lower and upper coordinate viewing limits of the current active X-axis. Here, it restricts viewing from 0 to 100.
Answer: (B)

Q15. Which command enables a dotted grey background grid on both coordinate axes?

(A) `plt.grid(axis='both', style='dots', color='grey')`
(B) `plt.background(grid=True, color='grey')`
(C) `plt.grid(visible=True, color='grey', linestyle=':')`
(D) `plt.grid(mode='dotted_grey')`

Sol. Q15. In Pyplot, the background grid is configured via `plt.grid(visible=True, color='grey', linestyle=':')`. The character `':'` denotes a dotted stroke pattern.
Answer: (C)